Q – 1 Tkinter form with CREATE,READ, UPDATE, DELETE Operation (Student Details).

- **Input :-**

```
from tkinter import *
from tkinter import ttk, messagebox
import sqlite3

# Database Connection
con = sqlite3.connect("student.db")
cur = con.cursor()

cur.execute("""
CREATE TABLE IF NOT EXISTS student (
   id INTEGER PRIMARY KEY AUTOINCREMENT,
   name TEXT NOT NULL,
   email TEXT NOT NULL,
   course TEXT NOT NULL
)
""")
con.commit()

root = Tk()
root.title("Student Details - CRUD")
root.geometry("700x500")
root.config(bg="lightblue")
```

```
selected_id = None

# Functions
def clear_fields():
    global selected_id
    selected_id = None
    name_entry.delete(0, END)
    email_entry.delete(0, END)
    course_entry.delete(0, END)
    tree.selection_remove(tree.selection())

def read_data():
    for item in tree.get_children():
        tree.delete(item)

    cur.execute("SELECT * FROM student")
    for row in cur.fetchall():
        tree.insert("", END, values=row)

def create_data():
    name = name_entry.get().strip()
    email = email_entry.get().strip()
    course = course_entry.get().strip()

    if not name or not email or not course:
```

```
        messagebox.showwarning(
            "Warning", "Please fill all fields"
        )
        return

    cur.execute(
        "INSERT INTO student (name, email, course) VALUES (?, ?, ?)",
        (name, email, course)
    )
    con.commit()
    read_data()
    clear_fields()
    messagebox.showinfo("Success", "Student Added Successfully")

def select_data(event):
    global selected_id
    selected = tree.focus()

    if selected:
        values = tree.item(selected, "values")
        selected_id = values[0]

        clear_entries_only()
        name_entry.insert(0, values[1])
        email_entry.insert(0, values[2])
        course_entry.insert(0, values[3])
```

```
def clear_entries_only():
    name_entry.delete(0, END)
    email_entry.delete(0, END)
    course_entry.delete(0, END)

def update_data():
    if selected_id is None:
        messagebox.showwarning(
            "Warning", "Please select a student first"
        )
        return

    name = name_entry.get().strip()
    email = email_entry.get().strip()
    course = course_entry.get().strip()

    if not name or not email or not course:
        messagebox.showwarning(
            "Warning", "Please fill all fields"
        )
        return

    cur.execute(
        "UPDATE student SET name=?, email=?, course=? WHERE id=?",
        (name, email, course, selected_id)
```

```
    )
    con.commit()
    read_data()
    clear_fields()
    messagebox.showinfo("Success", "Student Updated Successfully")

def delete_data():
    if selected_id is None:
        messagebox.showwarning(
            "Warning", "Please select a student first"
        )
        return

    answer = messagebox.askyesno(
        "Confirm", "Delete selected student?"
    )

    if answer:
        cur.execute(
            "DELETE FROM student WHERE id=?", (selected_id,)
        )
        con.commit()
        read_data()
        clear_fields()
        messagebox.showinfo("Success", "Student Deleted Successfully")
```

```
# Heading
Label(
   root, text="Student Details Form",
   font=("Arial", 18, "bold"), bg="lightblue"
).pack(pady=10)

# Form
form = Frame(root, bg="lightblue")
form.pack(pady=5)

Label(form, text="Name:", bg="lightblue").grid(
   row=0, column=0, padx=5, pady=5, sticky="e"
)
name_entry = Entry(form, width=30)
name_entry.grid(row=0, column=1, padx=5, pady=5)

Label(form, text="Email:", bg="lightblue").grid(
   row=1, column=0, padx=5, pady=5, sticky="e"
)
email_entry = Entry(form, width=30)
email_entry.grid(row=1, column=1, padx=5, pady=5)

Label(form, text="Course:", bg="lightblue").grid(
   row=2, column=0, padx=5, pady=5, sticky="e"
)
course_entry = Entry(form, width=30)
```

```
course_entry.grid(row=2, column=1, padx=5, pady=5)

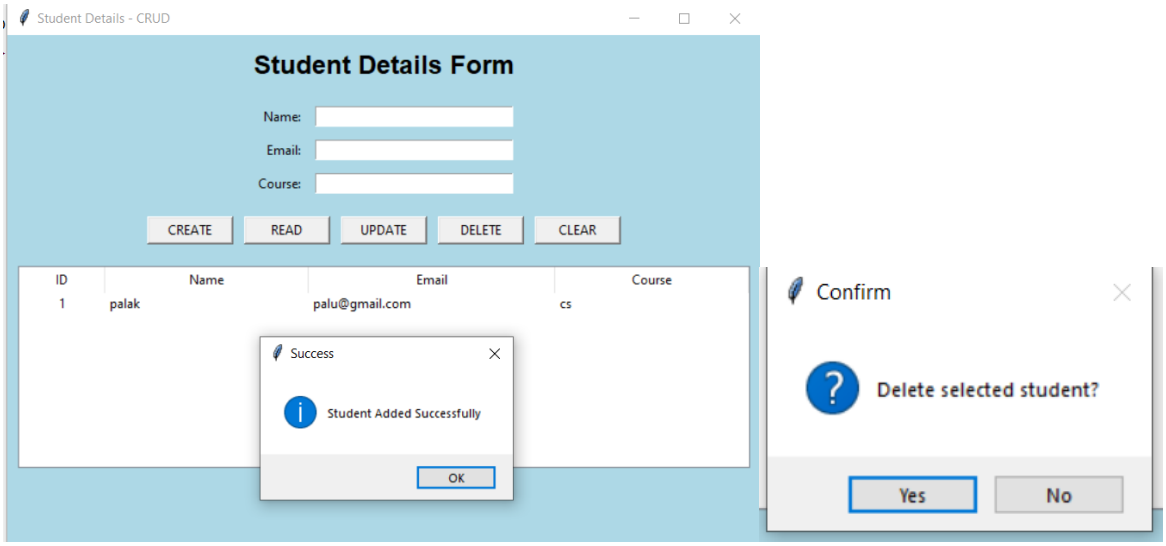
# Buttons
buttons = Frame(root, bg="lightblue")
buttons.pack(pady=10)

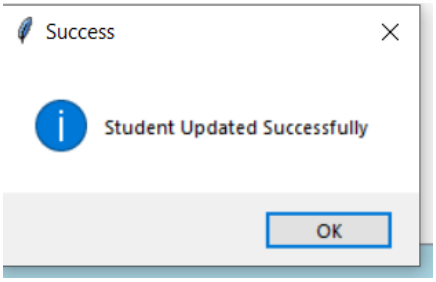
Button(buttons, text="CREATE", width=10,
    command=create_data).grid(row=0, column=0, padx=5)
Button(buttons, text="READ", width=10,
    command=read_data).grid(row=0, column=1, padx=5)
Button(buttons, text="UPDATE", width=10,
    command=update_data).grid(row=0, column=2, padx=5)
Button(buttons, text="DELETE", width=10,
    command=delete_data).grid(row=0, column=3, padx=5)
Button(buttons, text="CLEAR", width=10,
    command=clear_fields).grid(row=0, column=4, padx=5)

# Student Table
tree = ttk.Treeview(
  root,
  columns=("ID", "Name", "Email", "Course"),
  show="headings",
  height=8
)
tree.heading("ID", text="ID")
tree.heading("Name", text="Name")
```

```
tree.heading("Email", text="Email")
tree.heading("Course", text="Course")
tree.column("ID", width=50, anchor=CENTER)
tree.column("Name", width=160)
tree.column("Email", width=200)
tree.column("Course", width=150)
tree.pack(pady=10, padx=10, fill=X)
tree.bind("<<TreeviewSelect>>", select_data)
read_data()
root.mainloop()
con.close()
```

➢ Output :-





Q – 2 Tkinter form with CREATE,READ, UPDATE, DELETE Operation (Member Details).

- **Input :-**

```
from tkinter import *
from tkinter import ttk, messagebox
import sqlite3

# Database Connection
con = sqlite3.connect("member.db")
cur = con.cursor()

cur.execute("""
CREATE TABLE IF NOT EXISTS member (
   id INTEGER PRIMARY KEY AUTOINCREMENT,
   name TEXT NOT NULL,
   email TEXT NOT NULL,
   phone TEXT NOT NULL,
   address TEXT NOT NULL
)
""")
con.commit()

root = Tk()
root.title("Member Details - CRUD Operations")
root.geometry("800x550")
root.config(bg="lightblue")

selected_id = None

# Clear Form
def clear_fields():
   global selected_id
   selected_id = None
   name_entry.delete(0, END)
   email_entry.delete(0, END)
   phone_entry.delete(0, END)
```

```
    address_entry.delete(0, END)
    tree.selection_remove(tree.selection())

# READ Operation
def read_data():
    for item in tree.get_children():
        tree.delete(item)

    cur.execute("SELECT * FROM member")
    for row in cur.fetchall():
        tree.insert("", END, values=row)

# CREATE Operation
def create_data():
    name = name_entry.get().strip()
    email = email_entry.get().strip()
    phone = phone_entry.get().strip()
    address = address_entry.get().strip()

    if not name or not email or not phone or not address:
        messagebox.showwarning(
            "Warning", "Please fill all fields"
        )
        return

    cur.execute("""
        INSERT INTO member (name, email, phone, address)
        VALUES (?, ?, ?, ?)
    """, (name, email, phone, address))

    con.commit()
    read_data()
    clear_fields()
    messagebox.showinfo(
        "Success", "Member Added Successfully"
    )

# Select Member from Table
```

```
def select_data(event):
    global selected_id
    selected = tree.focus()

    if selected:
        values = tree.item(selected, "values")
        selected_id = values[0]

        name_entry.delete(0, END)
        email_entry.delete(0, END)
        phone_entry.delete(0, END)
        address_entry.delete(0, END)

        name_entry.insert(0, values[1])
        email_entry.insert(0, values[2])
        phone_entry.insert(0, values[3])
        address_entry.insert(0, values[4])

# UPDATE Operation
def update_data():
    if selected_id is None:
        messagebox.showwarning(
            "Warning", "Please select a member first"
        )
        return

    name = name_entry.get().strip()
    email = email_entry.get().strip()
    phone = phone_entry.get().strip()
    address = address_entry.get().strip()

    if not name or not email or not phone or not address:
        messagebox.showwarning(
            "Warning", "Please fill all fields"
        )
        return

    cur.execute("""
```

```
        UPDATE member
        SET name=?, email=?, phone=?, address=?
        WHERE id=?
    """, (name, email, phone, address, selected_id))

    con.commit()
    read_data()
    clear_fields()
    messagebox.showinfo(
        "Success", "Member Updated Successfully"
    )

# DELETE Operation
def delete_data():
    if selected_id is None:
        messagebox.showwarning(
            "Warning", "Please select a member first"
        )
        return

    answer = messagebox.askyesno(
        "Confirm", "Do you want to delete this member?"
    )

    if answer:
        cur.execute(
            "DELETE FROM member WHERE id=?",
            (selected_id,)
        )
        con.commit()
        read_data()
        clear_fields()
        messagebox.showinfo(
            "Success", "Member Deleted Successfully"
        )

# Heading
Label(
```

```
    root,
    text="Member Details Form",
    font=("Arial", 18, "bold"),
    bg="lightblue"
).pack(pady=10)

# Form Frame
form = Frame(root, bg="lightblue")
form.pack(pady=5)

Label(form, text="Name:", bg="lightblue").grid(
    row=0, column=0, padx=5, pady=5, sticky="e"
)
name_entry = Entry(form, width=35)
name_entry.grid(row=0, column=1, padx=5, pady=5)

Label(form, text="Email:", bg="lightblue").grid(
    row=1, column=0, padx=5, pady=5, sticky="e"
)
email_entry = Entry(form, width=35)
email_entry.grid(row=1, column=1, padx=5, pady=5)

Label(form, text="Phone:", bg="lightblue").grid(
    row=2, column=0, padx=5, pady=5, sticky="e"
)
phone_entry = Entry(form, width=35)
phone_entry.grid(row=2, column=1, padx=5, pady=5)

Label(form, text="Address:", bg="lightblue").grid(
    row=3, column=0, padx=5, pady=5, sticky="e"
)
address_entry = Entry(form, width=35)
address_entry.grid(row=3, column=1, padx=5, pady=5)

# Buttons
buttons = Frame(root, bg="lightblue")
buttons.pack(pady=10)
```

```
Button(
   buttons, text="CREATE", width=10,
   command=create_data
).grid(row=0, column=0, padx=5)

Button(
   buttons, text="READ", width=10,
   command=read_data
).grid(row=0, column=1, padx=5)

Button(
   buttons, text="UPDATE", width=10,
   command=update_data
).grid(row=0, column=2, padx=5)

Button(
   buttons, text="DELETE", width=10,
   command=delete_data
).grid(row=0, column=3, padx=5)

Button(
   buttons, text="CLEAR", width=10,
   command=clear_fields
).grid(row=0, column=4, padx=5)

# Member Records Table
tree = ttk.Treeview(
   root,
   columns=("ID", "Name", "Email", "Phone", "Address"),
   show="headings",
   height=8
)

tree.heading("ID", text="ID")
tree.heading("Name", text="Name")
tree.heading("Email", text="Email")
tree.heading("Phone", text="Phone")
tree.heading("Address", text="Address")
```

```
tree.column("ID", width=40, anchor=CENTER)
tree.column("Name", width=130)
tree.column("Email", width=180)
tree.column("Phone", width=110)
tree.column("Address", width=200)

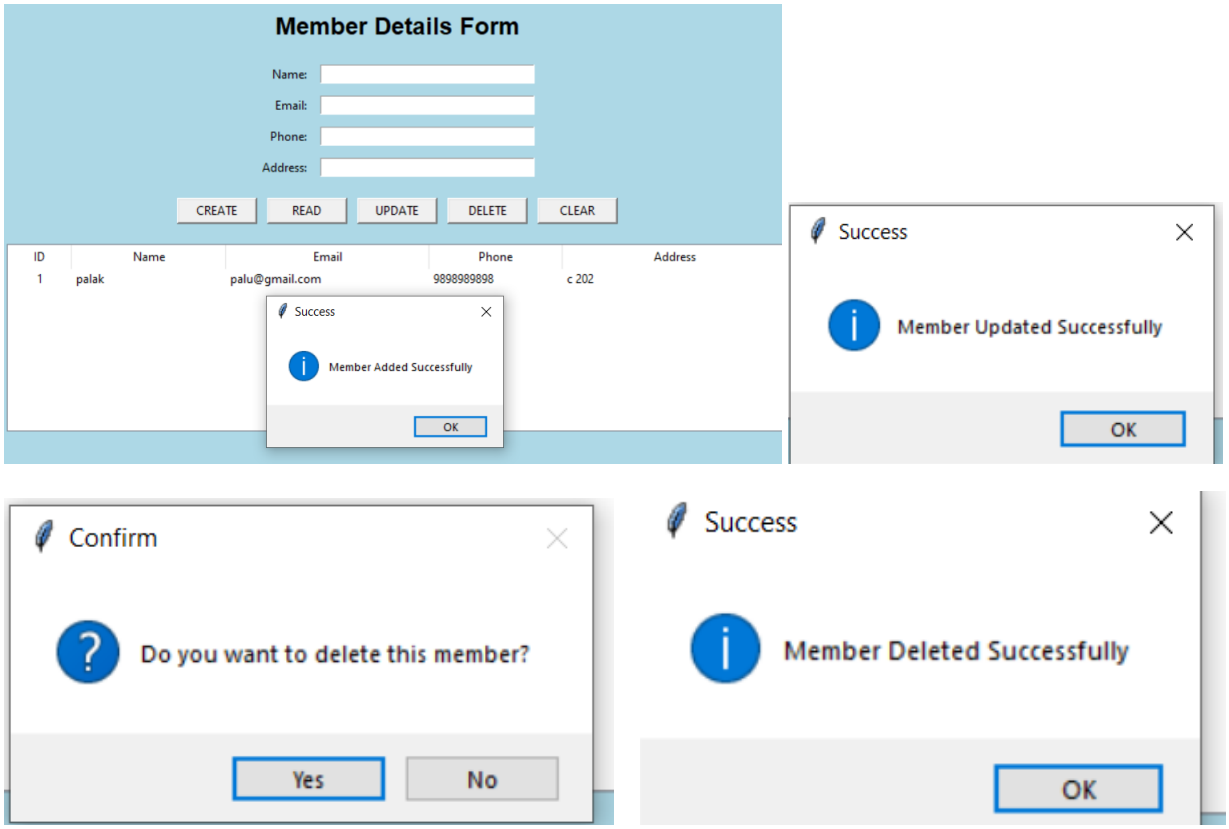
tree.pack(padx=10, pady=10, fill=X)
tree.bind("<<TreeviewSelect>>", select_data)

# Display Existing Records
read_data()

root.mainloop()

con.close()
```

➢ **Output :-**



Q – 3 Tkinter form with CREATE,READ, UPDATE, DELETE Operation (Patient Details).

- **Input :-**

```
from tkinter import *
from tkinter import ttk, messagebox
import sqlite3

# Database Connection
con = sqlite3.connect("patient.db")
cur = con.cursor()

cur.execute("""
CREATE TABLE IF NOT EXISTS patient (
   id INTEGER PRIMARY KEY AUTOINCREMENT,
   name TEXT NOT NULL,
   age INTEGER NOT NULL,
   gender TEXT NOT NULL,
   disease TEXT NOT NULL,
   phone TEXT NOT NULL
)
""")
con.commit()

root = Tk()
root.title("Patient Details - CRUD Operations")
root.geometry("850x570")
root.config(bg="lightblue")

selected_id = None

# Clear Form
def clear_fields():
   global selected_id
   selected_id = None
   name_entry.delete(0, END)
   age_entry.delete(0, END)
```

```
    gender_var.set("Male")
    disease_entry.delete(0, END)
    phone_entry.delete(0, END)
    tree.selection_remove(tree.selection())

# READ Operation
def read_data():
    for item in tree.get_children():
        tree.delete(item)

    cur.execute("SELECT * FROM patient")
    for row in cur.fetchall():
        tree.insert("", END, values=row)

# CREATE Operation
def create_data():
    name = name_entry.get().strip()
    age = age_entry.get().strip()
    gender = gender_var.get()
    disease = disease_entry.get().strip()
    phone = phone_entry.get().strip()

    if not name or not age or not disease or not phone:
        messagebox.showwarning(
            "Warning", "Please fill all fields"
        )
        return

    try:
        age_value = int(age)
        if age_value <= 0:
            raise ValueError
    except ValueError:
        messagebox.showerror(
            "Error", "Please enter a valid age"
        )
        return
```

```
    cur.execute("""
        INSERT INTO patient (name, age, gender, disease, phone)
        VALUES (?, ?, ?, ?, ?)
    """, (name, age_value, gender, disease, phone))

    con.commit()
    read_data()
    clear_fields()
    messagebox.showinfo(
        "Success", "Patient Added Successfully"
    )

# Select Patient from Table
def select_data(event):
    global selected_id
    selected = tree.focus()

    if selected:
        values = tree.item(selected, "values")
        selected_id = values[0]

        name_entry.delete(0, END)
        age_entry.delete(0, END)
        disease_entry.delete(0, END)
        phone_entry.delete(0, END)

        name_entry.insert(0, values[1])
        age_entry.insert(0, values[2])
        gender_var.set(values[3])
        disease_entry.insert(0, values[4])
        phone_entry.insert(0, values[5])

# UPDATE Operation
def update_data():
    if selected_id is None:
        messagebox.showwarning(
            "Warning", "Please select a patient first"
        )
```

```
    return

name = name_entry.get().strip()
age = age_entry.get().strip()
gender = gender_var.get()
disease = disease_entry.get().strip()
phone = phone_entry.get().strip()

if not name or not age or not disease or not phone:
    messagebox.showwarning(
        "Warning", "Please fill all fields"
    )
    return

try:
    age_value = int(age)
    if age_value <= 0:
        raise ValueError
except ValueError:
    messagebox.showerror(
        "Error", "Please enter a valid age"
    )
    return

cur.execute("""
    UPDATE patient
    SET name=?, age=?, gender=?, disease=?, phone=?
    WHERE id=?
""", (
    name, age_value, gender, disease, phone, selected_id
))

con.commit()
read_data()
clear_fields()
messagebox.showinfo(
    "Success", "Patient Updated Successfully"
)
```

```
# DELETE Operation
def delete_data():
    if selected_id is None:
        messagebox.showwarning(
            "Warning", "Please select a patient first"
        )
        return

    answer = messagebox.askyesno(
        "Confirm", "Do you want to delete this patient?"
    )

    if answer:
        cur.execute(
            "DELETE FROM patient WHERE id=?",
            (selected_id,)
        )
        con.commit()
        read_data()
        clear_fields()
        messagebox.showinfo(
            "Success", "Patient Deleted Successfully"
        )

# Heading
Label(
    root,
    text="Patient Details Form",
    font=("Arial", 18, "bold"),
    bg="lightblue"
).pack(pady=10)

# Form Frame
form = Frame(root, bg="lightblue")
form.pack(pady=5)

Label(form, text="Patient Name:", bg="lightblue").grid(
```

```
   row=0, column=0, padx=5, pady=5, sticky="e"
)
name_entry = Entry(form, width=30)
name_entry.grid(row=0, column=1, padx=5, pady=5)

Label(form, text="Age:", bg="lightblue").grid(
   row=1, column=0, padx=5, pady=5, sticky="e"
)
age_entry = Entry(form, width=30)
age_entry.grid(row=1, column=1, padx=5, pady=5)

Label(form, text="Gender:", bg="lightblue").grid(
   row=2, column=0, padx=5, pady=5, sticky="e"
)
gender_var = StringVar(value="Male")

Radiobutton(
   form, text="Male", variable=gender_var,
   value="Male", bg="lightblue"
).grid(row=2, column=1, sticky="w")

Radiobutton(
   form, text="Female", variable=gender_var,
   value="Female", bg="lightblue"
).grid(row=2, column=1, padx=90, sticky="w")

Label(form, text="Disease:", bg="lightblue").grid(
   row=3, column=0, padx=5, pady=5, sticky="e"
)
disease_entry = Entry(form, width=30)
disease_entry.grid(row=3, column=1, padx=5, pady=5)

Label(form, text="Phone:", bg="lightblue").grid(
   row=4, column=0, padx=5, pady=5, sticky="e"
)
phone_entry = Entry(form, width=30)
phone_entry.grid(row=4, column=1, padx=5, pady=5)
```

```
# Buttons
buttons = Frame(root, bg="lightblue")
buttons.pack(pady=10)

Button(
   buttons, text="CREATE", width=10,
   command=create_data
).grid(row=0, column=0, padx=5)

Button(
   buttons, text="READ", width=10,
   command=read_data
).grid(row=0, column=1, padx=5)

Button(
   buttons, text="UPDATE", width=10,
   command=update_data
).grid(row=0, column=2, padx=5)

Button(
   buttons, text="DELETE", width=10,
   command=delete_data
).grid(row=0, column=3, padx=5)

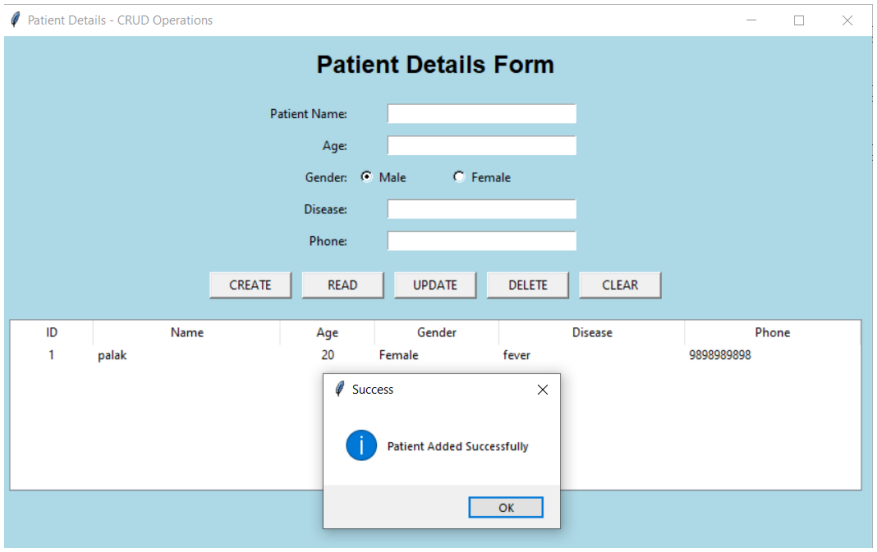
Button(
   buttons, text="CLEAR", width=10,
   command=clear_fields
).grid(row=0, column=4, padx=5)

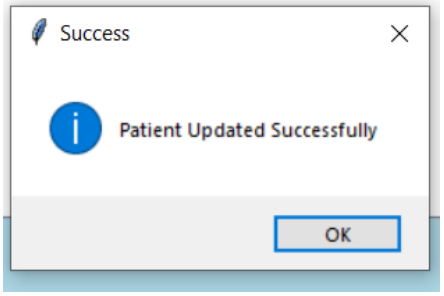
# Patient Records Table
tree = ttk.Treeview(
   root,
   columns=("ID", "Name", "Age", "Gender", "Disease", "Phone"),
   show="headings",
   height=7
)

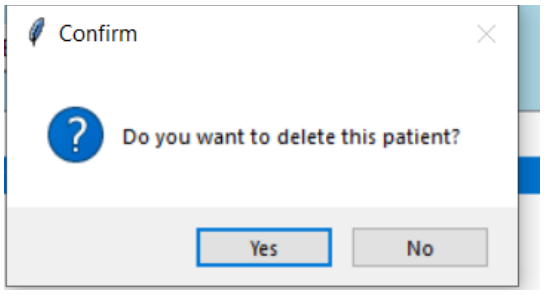
for col in ("ID", "Name", "Age", "Gender", "Disease", "Phone"):
```

```
    tree.heading(col, text=col)
tree.column("ID", width=40, anchor=CENTER)
tree.column("Name", width=140)
tree.column("Age", width=50, anchor=CENTER)
tree.column("Gender", width=80)
tree.column("Disease", width=140)
tree.column("Phone", width=130)
tree.pack(padx=10, pady=10, fill=X)
tree.bind("<<TreeviewSelect>>", select_data)
# Display Existing Records
read_data()
root.mainloop()
con.close()
```

➢ **Output :-**







Q - 4 Tkinter form with CREATE,READ, UPDATE, DELETE Operation (emp Details).

- **Input :-**

```
from tkinter import *
from tkinter import ttk, messagebox
import sqlite3

# Database Connection
con = sqlite3.connect("employee.db")
cur = con.cursor()

cur.execute("""
CREATE TABLE IF NOT EXISTS employee (
   id INTEGER PRIMARY KEY AUTOINCREMENT,
   name TEXT NOT NULL,
   department TEXT NOT NULL,
   salary REAL NOT NULL,
   phone TEXT NOT NULL
)
""")
con.commit()

root = Tk()
root.title("Employee Details - CRUD Operations")
root.geometry("850x550")
root.config(bg="lightblue")

selected_id = None

# Clear Form
def clear_fields():
   global selected_id
   selected_id = None
   name_entry.delete(0, END)
   dept_entry.delete(0, END)
   salary_entry.delete(0, END)
```

```
    phone_entry.delete(0, END)
    tree.selection_remove(tree.selection())

# READ Operation
def read_data():
    for item in tree.get_children():
        tree.delete(item)

    cur.execute("SELECT * FROM employee")
    for row in cur.fetchall():
        tree.insert("", END, values=row)

# CREATE Operation
def create_data():
    name = name_entry.get().strip()
    department = dept_entry.get().strip()
    salary = salary_entry.get().strip()
    phone = phone_entry.get().strip()

    if not name or not department or not salary or not phone:
        messagebox.showwarning(
            "Warning", "Please fill all fields"
        )
        return

    try:
        salary_value = float(salary)
        if salary_value < 0:
            raise ValueError
    except ValueError:
        messagebox.showerror(
            "Error", "Please enter a valid salary"
        )
        return

    cur.execute("""
        INSERT INTO employee (name, department, salary, phone)
        VALUES (?, ?, ?, ?)
```

```
    """, (name, department, salary_value, phone))

    con.commit()
    read_data()
    clear_fields()
    messagebox.showinfo(
        "Success", "Employee Added Successfully"
    )

# Select Employee from Table
def select_data(event):
    global selected_id
    selected = tree.focus()

    if selected:
        values = tree.item(selected, "values")
        selected_id = values[0]

        name_entry.delete(0, END)
        dept_entry.delete(0, END)
        salary_entry.delete(0, END)
        phone_entry.delete(0, END)

        name_entry.insert(0, values[1])
        dept_entry.insert(0, values[2])
        salary_entry.insert(0, values[3])
        phone_entry.insert(0, values[4])

# UPDATE Operation
def update_data():
    if selected_id is None:
        messagebox.showwarning(
            "Warning", "Please select an employee first"
        )
        return

    name = name_entry.get().strip()
    department = dept_entry.get().strip()
```

```
    salary = salary_entry.get().strip()
    phone = phone_entry.get().strip()

    if not name or not department or not salary or not phone:
        messagebox.showwarning(
            "Warning", "Please fill all fields"
        )
        return

    try:
        salary_value = float(salary)
        if salary_value < 0:
            raise ValueError
    except ValueError:
        messagebox.showerror(
            "Error", "Please enter a valid salary"
        )
        return

    cur.execute("""
        UPDATE employee
        SET name=?, department=?, salary=?, phone=?
        WHERE id=?
    """, (name, department, salary_value, phone, selected_id))

    con.commit()
    read_data()
    clear_fields()
    messagebox.showinfo(
        "Success", "Employee Updated Successfully"
    )

# DELETE Operation
def delete_data():
    if selected_id is None:
        messagebox.showwarning(
            "Warning", "Please select an employee first"
        )
```

```
        return

    answer = messagebox.askyesno(
        "Confirm", "Do you want to delete this employee?"
    )

    if answer:
        cur.execute(
            "DELETE FROM employee WHERE id=?",
            (selected_id,)
        )
        con.commit()
        read_data()
        clear_fields()
        messagebox.showinfo(
            "Success", "Employee Deleted Successfully"
        )

# Heading
Label(
    root,
    text="Employee Details Form",
    font=("Arial", 18, "bold"),
    bg="lightblue"
).pack(pady=10)

# Form Frame
form = Frame(root, bg="lightblue")
form.pack(pady=5)

Label(form, text="Employee Name:", bg="lightblue").grid(
    row=0, column=0, padx=5, pady=5, sticky="e"
)
name_entry = Entry(form, width=30)
name_entry.grid(row=0, column=1, padx=5, pady=5)

Label(form, text="Department:", bg="lightblue").grid(
    row=1, column=0, padx=5, pady=5, sticky="e"
```

```
)
dept_entry = Entry(form, width=30)
dept_entry.grid(row=1, column=1, padx=5, pady=5)

Label(form, text="Salary:", bg="lightblue").grid(
   row=2, column=0, padx=5, pady=5, sticky="e"
)
salary_entry = Entry(form, width=30)
salary_entry.grid(row=2, column=1, padx=5, pady=5)

Label(form, text="Phone:", bg="lightblue").grid(
   row=3, column=0, padx=5, pady=5, sticky="e"
)
phone_entry = Entry(form, width=30)
phone_entry.grid(row=3, column=1, padx=5, pady=5)

# Buttons
buttons = Frame(root, bg="lightblue")
buttons.pack(pady=10)

Button(
   buttons, text="CREATE", width=10,
   command=create_data
).grid(row=0, column=0, padx=5)

Button(
   buttons, text="READ", width=10,
   command=read_data
).grid(row=0, column=1, padx=5)

Button(
   buttons, text="UPDATE", width=10,
   command=update_data
).grid(row=0, column=2, padx=5)

Button(
   buttons, text="DELETE", width=10,
   command=delete_data
```

```
).grid(row=0, column=3, padx=5)

Button(
   buttons, text="CLEAR", width=10,
   command=clear_fields
).grid(row=0, column=4, padx=5)

# Employee Records Table
tree = ttk.Treeview(
   root,
   columns=("ID", "Name", "Department", "Salary", "Phone"),
   show="headings",
   height=8
)

tree.heading("ID", text="ID")
tree.heading("Name", text="Name")
tree.heading("Department", text="Department")
tree.heading("Salary", text="Salary")
tree.heading("Phone", text="Phone")

tree.column("ID", width=40, anchor=CENTER)
tree.column("Name", width=150)
tree.column("Department", width=150)
tree.column("Salary", width=120)
tree.column("Phone", width=130)

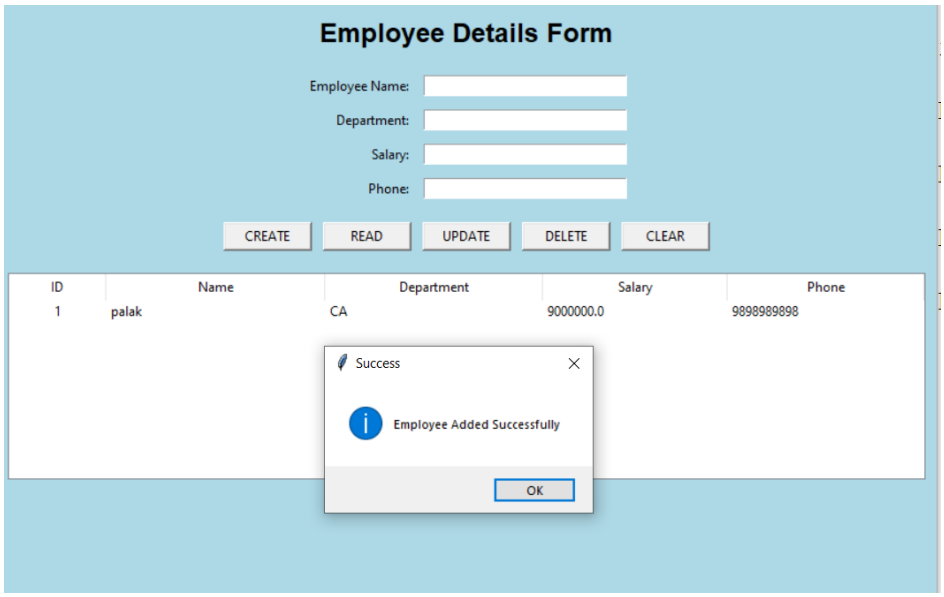
tree.pack(padx=10, pady=10, fill=X)
tree.bind("<<TreeviewSelect>>", select_data)

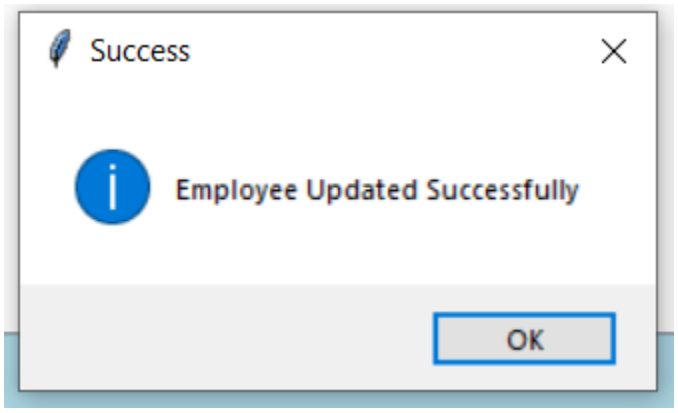
# Display Existing Records
read_data()

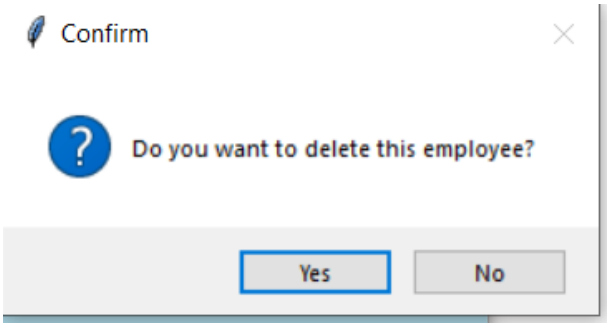
root.mainloop()

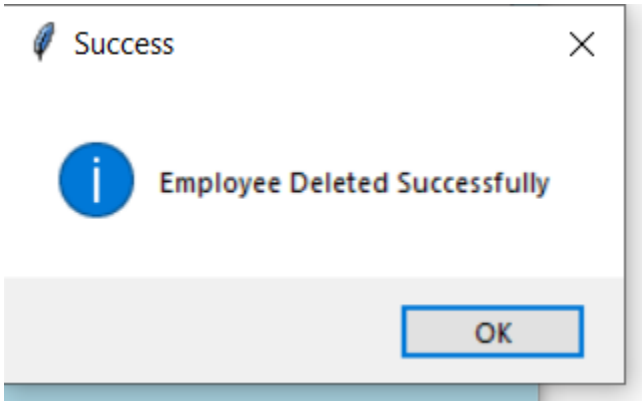
con.close()
```

- **Output :-**


Employee Details Form
Employee Name:
Department:
Salary:
Phone:
CREATE
READ
UPDATE
DELETE
CLEAR
ID
Name
Department
Salary
Phone
1
palak
CA
9000000.0
9898989898
Success
Employee Added Successfully
OK


Success
Employee Updated Successfully
OK


Confirm
Do you want to delete this employee?
Yes
No


Success
Employee Deleted Successfully
OK